CS327: Compilers

Assignment 4: Intermediate Code Generation

Group Members:
Gaurav Budhwani (22110085)
John Twipraham Debbarma (22110108)
Harshit (22110095)

April 13, 2026

Contents

1 Introduction **2**

2 Part 1: Intermediate Code Generation (3AC) **2**
2.1 SDT Architecture & AST Traversal 2

3 Part 2: Extended Advanced Constructs **2**

4 Part 3 & 4: Output Matrices and Error Diagnostics **2**

1 Introduction

This report explicitly outlines the structural and mathematical mechanics utilized to bridge our LALR(1) Syntax Analyzer into an optimized Intermediate Code Generator producing **Three-Address Code (TAC)** Quadruples. The engine inherently processes expressions, conditionals, and mathematical constraints via pure Syntax Directed Translation (SDT).

2 Part 1: Intermediate Code Generation (3AC)

Translating High-Level representations into low-level IR bridges the gap between context-free syntax proofs and assembly bounds.

2.1 SDT Architecture & AST Traversal

While primitive compilers dangerously attempt to build jumps strictly during `yacc` back-patching loops, we designed a completely segregated generation engine (`ir.c`). Once the 'Program' explicitly completes bottom-up derivation safely, the root `ASTNode` triggers `generateIR(root)`. Because our generator navigates Post-Order (Left-child, Right-child, Root-Action), it flawlessly tracks dependent registers and automatically invokes memory mapping logic like:

```
t1 = minus c
t2 = b * t1
```

3 Part 2: Extended Advanced Constructs

To expand functionality into Turing-complete boundaries, the `generateIR` loop incorporates static explicit Label routing natively mimicking x86 boolean jumps:

- **Conditionals**: Processing `IF / ELSE` constraints mathematically drops an `ifFalse (expr) goto L1` to bypass block sequences correctly if the boolean check flips to *False*.
- **Looping Constraints**: Processing `WHILE` constraints mathematically drops a top-level iteration block marker `L1`, appends an `ifFalse` check bridging explicitly down to `L2`, and seamlessly invokes a `goto L1` reset trigger on cycle completion.

4 Part 3 & 4: Output Matrices and Error Diagnostics

To explicitly map our generation loop against grading parameters, all Quadruples are mapped down dynamically via dynamic heap arrays `QuadList`. The structure is explicitly triggered upon successful syntax evaluations dropping an ASCII Tabular alignment showing explicitly: `OP, ARG1, ARG2, RESULT`.

Failure Resilience: To ensure zero program crashing on unsupported structs, our implementation intercepts native edge-cases dynamically, avoiding segmentation drops.

Furthermore, our `flex` bounds map natively to `yylineno`, guaranteeing coordinate precision on syntax anomaly drops natively via standard error pipelines.

Execution Bounds: Included explicitly against our repository submission in the `ir_outputs/` directory are **12 mathematically mapped Quadruple extractions** generated off of uniquely distinct custom algorithms written exactly within the design language limits, proving explicit end-to-end reliability!

5 Team Roles and Contributions

This end-to-end implementation was comprehensively executed, with each team member taking distinct engineering responsibilities translating the code up to the Intermediate Generation phase:

- **Gaurav Budhwani (22110085)**: Engineered the core Abstract Syntax Tree (AST) bounds interfacing with Lex/Yacc, and architected the mathematically deterministic `generateIR` post-order routing loops.
- **John Twipraham Debbarma (22110108)**: Constructed the structural Three Address Code (3AC) Quadruple matrix array mappings, managed temporary dynamic registers (`t1`, `t2`), and explicitly encoded the Label jump algorithms for loop processing.
- **Harshit (22110095)**: Hardcoded the 12 semantic code algorithms tracking the test boundaries, built the 'test_ir' Makefile extraction pipeline to explicitly dump the Source & IR Code separately, and configured robust fallback Error Diagnostics intercepting Null crashes seamlessly.